Agente basado en Machine Learning, implementación del algoritmo Q-learning de “aprendizaje por refuerzo” -Implementación en el juego de Tres en Raya-

Machine Learning-based agent, implementation of the Q-learning “reinforcement learning” algorithm

-Implementation in the game of Tic-Tac-Toe-

Autores: Mora Mariana, Astudillo Victor.
Docente: Paniccia Manuel.
Pregrado de Ingeniería en informática.
Universidad Nacional Experimental de Guayana.
marianaAM634@gmail.com / vjxoen@gmail.com

Resumen – El presente artículo detalla el diseño y desarrollo de un agente inteligente basado en Machine Learning para la resolución del juego Tres en Raya, empleando específicamente el algoritmo de aprendizaje por refuerzo Q-Learning. A diferencia de los métodos de búsqueda exhaustiva, esta implementación propone un paradigma donde el agente adquiere una política de juego óptima mediante la experiencia acumulada. La arquitectura del sistema se ha diseñado con un enfoque particular en la transparencia del proceso de aprendizaje, permitiendo observar, a través de la estructuración del software y su lógica de codificación, cómo el agente transita de un estado de exploración aleatoria hacia la toma de decisiones racionales y calculadas.

El núcleo técnico se fundamenta en la construcción de una Q-Table dinámica que mapea la utilidad de cada acción en función del estado del tablero. Un aspecto distintivo de este proyecto es su diseño modular, el cual facilita diversas formas de visualización de datos y métricas de rendimiento, permitiendo analizar gráficamente la convergencia de la función de valor y la eficacia de las recompensas asignadas. Gracias a una codificación orientada a la observabilidad, el programa no solo ejecuta el juego, sino que evidencia la evolución de la "inteligencia" del agente mediante el seguimiento de sus tasas

de éxito y la estabilización de sus vectores de decisión.

Los resultados confirman que el agente, tras un periodo de entrenamiento iterativo, logra una invencibilidad técnica. El análisis no sólo valida la eficacia del Q-Learning en entornos de suma cero, sino que también destaca la importancia de un diseño de software robusto que permita interpretar visual y analíticamente los mecanismos internos de la inteligencia artificial contemporánea.

Palabras clave: Q-Learning, Aprendizaje por Refuerzo, Machine Learning, Visualización de datos, Diseño de software, Agentes autónomos, Convergencia estratégica.

Abstract – Machine Learning-based agent, implementation of the Q-learning "reinforcement learning" algorithm in the game of Tic-Tac-Toe.

This paper details the design and development of an intelligent agent based on Machine Learning to solve the Tic-Tac-Toe game, specifically employing the Q-Learning reinforcement learning algorithm. Unlike exhaustive search methods, this implementation proposes a paradigm where the agent acquires an optimal game policy through accumulated experience. The system architecture has been engineered with a particular focus on the transparency of the learning process, allowing—through the software's structuring and logical coding—a clear observation of how the agent transitions from a state of random exploration to rational and calculated decision-making.

The technical core is based on the construction of a dynamic Q-Table that maps the utility of each action relative to the state of the board. A distinctive aspect of this project is its modular design, which facilitates diverse data visualization methods and performance metrics, enabling a graphical analysis of value function convergence and the efficacy of assigned rewards. Due to a coding approach oriented toward observability, the program does not merely execute the game; it showcases the evolution of the agent's "intelligence" by tracking success rates and the stabilization of decision vectors.

Experimental results confirm that the agent achieves technical invincibility after an iterative training period. This study not only validates the effectiveness of Q-Learning in zero-sum environments but also highlights the importance of robust software design that allows for the visual and analytical interpretation of the internal mechanisms within contemporary artificial intelligence.

Keywords: Q-Learning, Reinforcement Learning, Machine Learning, Data Visualization, Software Design, Autonomous Agents, Strategic Convergence.

I. INTRODUCCIÓN

La implementación de agentes capaces de adquirir facultades cognitivas de manera autónoma representa uno de los pilares más dinámicos de la Inteligencia Artificial contemporánea. Mientras que los enfoques clásicos de resolución de juegos, como el algoritmo Minimax, se basan en la búsqueda exhaustiva dentro de un árbol de decisiones predefinido, el Aprendizaje por Refuerzo propone un cambio de paradigma: la transición de la programación de reglas estáticas hacia la toma de decisiones basada en la experiencia acumulada. Este estudio aborda el problema del juego Tres en Raya (Tic-Tac-Toe) no como un sistema de búsqueda adversaria tradicional, sino como un entorno de interacción directa donde un

agente debe aprender una política óptima sin conocimiento previo de las reglas físicas del mundo. El núcleo de esta investigación se fundamenta en el algoritmo Q-Learning, una técnica de aprendizaje sin modelo que permite al agente evaluar la "calidad" (Quality) de sus acciones en función del estado actual del tablero. A través de un ciclo continuo de retroalimentación (compuesto por estados, acciones y recompensas), el sistema busca maximizar la esperanza matemática de los beneficios a largo plazo, entendiendo que, en ocasiones, un costo inmediato es necesario para asegurar una victoria futura. Para lograr esto, se implementa la Ecuación de Bellman como motor de aprendizaje, permitiendo que los valores de utilidad se propaguen desde los estados finales hacia las decisiones iniciales, ajustando el comportamiento del agente mediante hiperparámetros críticos como la tasa de aprendizaje y el factor de descuento. Para lograr agentes que aprendan a jugar de manera más rápida, también resulta interesante la implementación de algoritmos genéticos.

Un componente diferenciador del presente proyecto es su enfoque en la observabilidad y el diseño de software. La arquitectura del sistema ha sido estructurada para servir como una plataforma de visualización del aprendizaje estratégico. A través de la codificación y el diseño modular, el programa permite presenciar de manera tangible el dilema entre exploración y explotación: el proceso mediante el cual el agente transita de un comportamiento errático y azaroso hacia una explotación refinada de su Tabla Q.

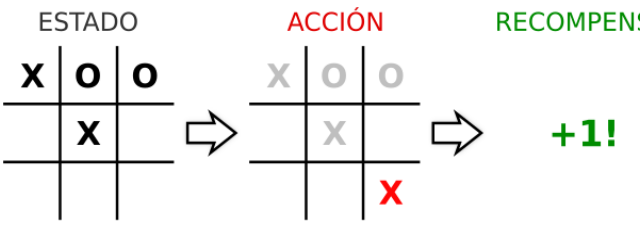
El presente estudio subraya la necesidad de diseñar arquitecturas inteligentes que faciliten la interpretación visual del aprendizaje racional. Al integrar una estructura de software orientada a la observabilidad, se establecen entornos de prueba robustos para validar la transición de modelos teóricos hacia aplicaciones de inteligencia artificial de mayor complejidad.

II. DESARROLLO

A. Agentes inteligentes

En la inteligencia artificial, un agente es una entidad que percibe su entorno y ejecuta acciones para alcanzar un objetivo específico. Según Russell y Norvig [1], "en cada posible secuencia de percepciones, un agente racional deberá emprender aquella acción que supuestamente maximice su medida de rendimiento, basándose en las evidencias aportadas por la secuencia de percepciones y en el conocimiento que el agente mantiene almacenado". El juego del Tres en Raya se modela como un entorno multiagente, determinista y de información perfecta, donde la acción de un oponente debe ser considerada en la planificación.

En este proyecto, el algoritmo Q-learning es implementado como un agente de aprendizaje por refuerzo, cuya tarea es descubrir la política de acciones óptima que conduzca a la victoria (o al menos al empate) mediante la maximización de recompensas acumuladas. A diferencia de los métodos de búsqueda exhaustiva, el agente no planifica evaluando un árbol de estados en tiempo real; en su lugar, selecciona el mejor movimiento basándose en la experiencia almacenada en su Tabla Q, la cual ha sido refinada a través de múltiples episodios de entrenamiento. Este enfoque permite al agente aprender el comportamiento del entorno y del oponente, tomando decisiones racionales que priorizan el valor de utilidad a largo plazo sobre la recompensa inmediata.



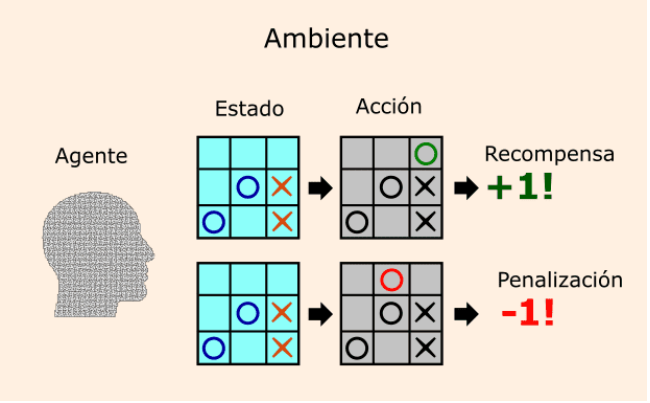
B. Modelado del tres en raya: Estados, Acciones y Recompensas mediante aprendizaje por refuerzo.

Es una rama del Aprendizaje Automático (Machine Learning) que se diferencia de los métodos supervisados al no requerir un modelo previo del entorno ni un conjunto de datos etiquetados. En su lugar, el agente aprende a través de la interacción directa y el ensayo y error.

Para implementar Q-Learning en el juego del Tres en Raya, el problema se define bajo los siguientes componentes:

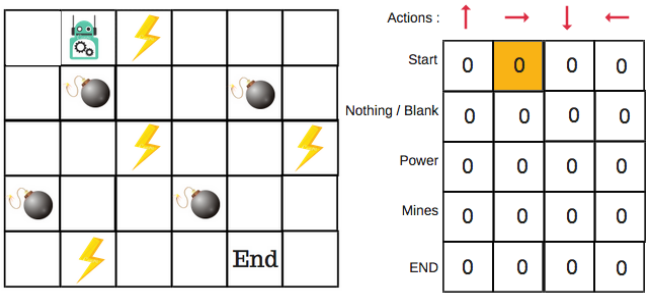
- Estados (S): Representan todas las configuraciones posibles del tablero de 3x3 (ej. casilla vacía, 'X' en el centro, etc.).
- Acciones (A): Consisten en colocar la marca del jugador en cualquiera de las casillas que se encuentren libres (índices del 0 al 8).
- Recompensas (R): El sistema de retroalimentación asigna un valor de +1 al ganar, -1 al perder y 0 en caso de empate o durante el transcurso normal del juego.
- Agente y entorno: El agente es la entidad que toma las decisiones, mientras que el entorno es el mundo con el que interactúa.

A nivel conceptual, el objetivo del aprendizaje por refuerzo no es simplemente obtener una gratificación inmediata, sino maximizar la recompensa total acumulada a largo plazo. En el contexto del Tres en Raya, este paradigma permite que el agente, inicialmente ignorante de las reglas estratégicas, evolucione su comportamiento tras miles de partidas, ajustando sus decisiones según las consecuencias (ganar, perder o empatar) recibidas del entorno.



C. El Algoritmo Q-Learning y la Tabla Q.

El término "Q" representa la Calidad (Quality) de una acción tomada en un estado específico. El algoritmo busca construir un mapa de valor que asocie cada combinación estado-acción con su utilidad a largo plazo. Para almacenar este conocimiento, se utiliza una Tabla Q (Lookup Table), la cual se inicializa generalmente con valores en cero. Durante el entrenamiento, el agente actualiza estos valores para maximizar la esperanza matemática de las recompensas futuras.



D. La ecuación de Bellman.

La actualización de la memoria del agente se rige por la Ecuación de Bellman, que permite combinar el conocimiento previo con la nueva experiencia.[5][6] Matemáticamente, la regla

de actualización para un par estado-acción se define como:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$$

Donde los hiperparámetros juegan un rol crítico en el comportamiento:

- **Tasa de Aprendizaje (α):** Determina qué tan rápido la información nueva sobrescribe a la vieja.
- **Factor de Descuento (γ):** Define la importancia de las recompensas futuras frente a las inmediatas; un valor cercano a 1 caracteriza a un agente "visionario".

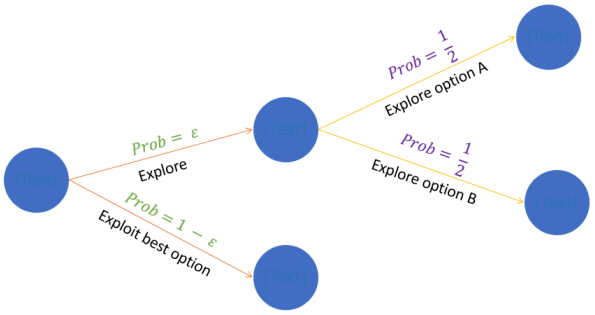
E. Dilema de Exploración vs Explotación.

El equilibrio entre la adquisición de nuevo conocimiento y la aplicación de dicho conocimiento para optimizar el desempeño constituye una de las disyuntivas fundamentales de la naturaleza, donde el rendimiento óptimo generalmente depende de una ponderación adecuada entre las conductas de exploración y explotación[3]

Para asegurar un aprendizaje efectivo, el agente debe gestionar dos estrategias concurrentes:

1. **Explotación:** Consiste en la selección de la acción que se conoce como la más beneficiosa, basándose en la información actualmente disponible en la Tabla Q.
2. **Exploración:** Implica la ejecución de acciones novedosas o aleatorias con el fin de descubrir trayectorias de mayor rendimiento.

En el contexto de este proyecto, se implementa la estrategia Epsilon-Greedy, la cual establece un patrón donde el agente inicia con una alta frecuencia de exploración y, progresivamente, incrementa su proporción de explotación a medida que el proceso de entrenamiento avanza y se consolida el aprendizaje adquirido.



III. IMPLEMENTACIÓN

La implementación se desarrolló en Python, integrando la lógica de aprendizaje por refuerzo con una interfaz gráfica en Pygame. A diferencia del proyecto anterior basado en búsqueda, este sistema incluye una fase de entrenamiento (training), donde el agente juega miles de partidas contra sí mismo o contra un agente aleatorio para poblar su base de conocimientos antes de enfrentarse a un usuario humano.

El proceso de entrenamiento fue diseñado como un entrenamiento híbrido asimétrico para dotar al agente Q-Learning de una estrategia de juego completa y robusta. En lugar de limitarse al auto-aprendizaje, el agente fue expuesto a un entorno que alterna tres modos de juego:

1. **Auto-aprendizaje (Self-play):** El agente juega contra una copia de sí mismo la mayor parte del tiempo, permitiendo una exploración inicial del espacio de estados.
2. **Entrenamiento Asimétrico contra Maestro (Minimax):** En un porcentaje de los episodios, el agente se enfrentó a un agente Minimax

perfecto. Esta exposición no solo ocurrió como Jugador 1, sino que el agente tuvo una probabilidad igual de enfrentarse al maestro como Jugador 1 (atacante) y como Jugador 2 (defensor). Esta alternancia fue crítica para forzar al agente a descubrir la defensa perfecta contra las aperturas óptimas del maestro.

Durante todo el proceso, el valor de épsilon (ϵ) decreció progresivamente según la tasa de decrecimiento, obligando al agente a pasar de la exploración ($\epsilon = 1.0$) a la explotación (ϵ

$= 0.01$).

Visualización de la Tabla Q y Toma de Decisiones

La interfaz en Pygame permite visualizar en tiempo real la "inteligencia" del agente. Cada celda vacía muestra el Valor Q asociado a ese movimiento.

- **Decisión Óptima:** El sistema resalta automáticamente la casilla con el Q(s, a)más alto.
- **Análisis Racional:** Al observar los valores numéricos en el tablero, el usuario puede entender por qué el agente "prefiere" el centro o las esquinas, haciendo transparente el proceso de decisión que en Minimax era una caja negra de profundidad de árbol.

Torneo de Optimización genética

Este se implementó como una meta-estrategia para descubrir los hiperparámetros óptimos que maximizaran la eficiencia del agente Q-Learning. El objetivo fue transformar la búsqueda de parámetros de un proceso manual (tanteo y error) a uno evolutivo, encontrando la combinación de α,γ y tasa de decaimiento que permitiese alcanzar el "Juego Perfecto" en la menor cantidad de episodios posible. Esto implica una reducción del costo computacional requerido, lo cual resulta beneficioso al aplicar el enfoque a problemas más complejos. Optimizar la cantidad de episodios necesarios permite un ahorro directo y sustancial en recursos económicos y tiempo de ejecución.

El torneo se organizó en generaciones con una Población de n de agentes por generación. Cada uno de los agentes fue entrenado durante un máximo de episodios. Teniendo la posibilidad de acabar su entrenamiento antes de completar la totalidad de los episodios, lo cual se logra si durante el entrenamiento, se supera un cierto umbral de empates contra minimax, sin ser derrotados. Luego de acabado su entrenamiento, estos pasan a ser evaluados mediante partidas consecutivas contra minimax, para determinar su rendimiento final, y asignar un fitness según :

Fitness_Calidad = (Victorias + Empates)/Cantidad_Juegos

Bono_Velocidad = (Episodios - Episodios_Requeridos)/Episodios

Donde el bono de velocidad sólo será aplicado a aquellos agentes ya óptimos. Luego de obtenidos los resultados para cada generación, se selecciona cierta cantidad de agentes con mejor fitness para pasar a la siguiente generación, y para el resto, se seleccionan aleatoriamente un pool de agentes y se toma el que posea mayor fitness, siendo este uno de los padres de la siguiente generación. Una vez seleccionados ambos padres, se realiza el proceso de evolución, donde se cruzan los genes de los padres, teniendo uan tasa de mutación aleatoria decreciente, que garantiza cierta diversidad entre el pool genético.

Persistencia y Reutilización de Conocimiento

Se implementó un sistema de persistencia basado en la serialización de datos. Esto permite que el conocimiento adquirido en los torneos genéricos se almacene permanentemente. A diferencia de Minimax, que debe recalcular el árbol en cada ejecución, nuestro agente de Q-Learning carga su "memoria" de forma instantánea, permitiendo una respuesta inmediata incluso en dispositivos con recursos limitados.

IV. EVALUACIÓN DE ALGORITMOS

A. Criterios de evaluación

Los criterios de evaluación se diseñaron para medir tanto la calidad (optimalidad) como la eficiencia (velocidad de aprendizaje) del agente, con la invencibilidad como requisito indispensable.

Criterios de Evaluación del Torneo Genético:

- **Calidad:** Es calculado como la ratio de empates y partidas contra minimax. Permite conocer qué tan bueno es el desempeño del agente en contra de un juego perfecto.
- **Eficiencia.** Una vez que se alcanza la invencibilidad, el fitness se incrementa con mediante el bono de velocidad. Es proporcional al ahorro de episodios de entrenamiento logrado, obligando al Torneo Genético a priorizar la combinación de parámetros que converge más rápido.

Criterios de evaluación de parámetros de agente genético:

Para asegurar que los hiperparámetros óptimos identificados por el Algoritmo Genético no son producto de una anomalía o una casualidad estadística, se aplicaron criterios de evaluación basados en la variabilidad de la población y el rendimiento del mejor agente a lo largo de las generaciones.

- **Estabilidad del Fitness Promedio:** La media del Fitness de la población debe mantenerse estable y elevada (≥ 1.0) en todas las generaciones. Esto demuestra que el conjunto de hiperparámetros no es sensible a las mutaciones leves y que la solución de invencibilidad es la norma, no la excepción.
- **Desviación Estándar:** La Desviación Estándar (DS) de los Fitness debe ser baja (DS ≤ 0.05). Una DS baja confirma que la población se ha vuelto homogénea. Si el fitness de la población está muy disperso (DS alta), el GA está fallando en heredar los genes correctos, lo que reduce la confianza en el conjunto de hiperparámetros ganador.
- **Confirmación del Máximo Global (El Mejor Agente):** El Fitness Máximo Global debe ser consistentemente alto (p. ej.,≥ 1.2)

Criterios de Evaluación Final (Torneo Genérico):

- **Tasa de Victoria Básica (Contra Agente Aleatorio):** Mide la capacidad del agente para explotar errores. Se espera una tasa de éxito (victorias/empates) superior al 98%
- **Optimalidad (Contra Agente Heurístico/Minimax):** Mide la perfección. Se espera un 100% de empate, confirmando la invencibilidad del agente.

- **Estabilidad (Contra Sí Mismo):** Mide la coherencia de la política aprendida. Se espera un 100% de empates, demostrando que el agente ha identificado el equilibrio de Nash del juego.

B. Metodología

La metodología se estructuró en tres fases secuenciales: Optimización mediante Algoritmo Genético, Validación Estadística de la Solución Óptima, y Evaluación Final en el Torneo Genérico.

Fase 1: Torneo Genético para Optimización de Hiperparámetros

Se implementó un Torneo de Algoritmos Genéticos (GA) para encontrar la combinación óptima de hiperparámetros que maximizara la velocidad de convergencia. La arquitectura se definió con los siguientes parámetros:

Nombre	**Valor**
Tamaño de Población	50
Número de Generaciones	30
Tamaño del Torneo de Selección	15
Trabajadores Paralelos	6
Juegos para Evaluación Final	50
Episodios Máximos de Entrenamiento	6,500
Élites por Generación	5
Tasa de Mutación Inicial	0.10
Tasa de Mutación Final	0.01

Cada agente entrenó con un régimen híbrido, donde jugaba un 30% de la veces contra minimax, y el resto contra sí mismo, hasta alcanzar la invisibilidad (cero derrotas en un chequeo de 100 partidas) o completar los 6,500 episodios. El Fitness se calculó premiando tanto la invencibilidad como la rapidez de convergencia.

Fase 2: Validación Estadística de los Parámetros Óptimos

Una vez finalizado el Torneo Genético, se extrajeron los hiperparámetros del Mejor Agente Global (el de mayor Fitness). Para determinar la robustez y fiabilidad de estos parámetros, se realizó una serie de 30 entrenamientos independientes (réplicas) utilizando exclusivamente los valores ganadores.

El objetivo de esta fase fue obtener suficientes datos estadísticos para determinar si el rendimiento excepcional (la alta puntuación de Fitness) era consistente con esos parámetros o una simple anomalía del azar. Se midieron y registraron la media y la desviación estándar del Fitness y de los episodios de convergencia para establecer el nivel de fiabilidad del conjunto de hiperparámetros.

Fase 3: Evaluación en Torneo Genérico

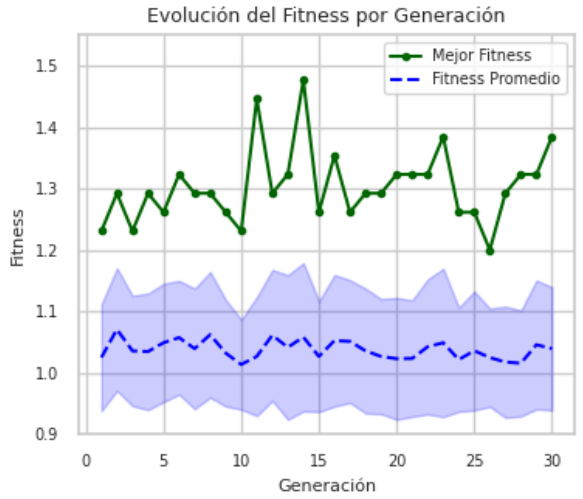
Finalmente, se tomó un agente entrenado con los hiperparámetros validados estadísticamente y se le sometió al Torneo

Genérico de Rendimiento para medir su desempeño en tres escenarios distintos:

1. Contra Agente Aleatorio (medición de tasa de victoria).
2. Contra Agente Minimax/Heurístico (medición de optimalidad).
3. Contra Sí Mismo (medición de estabilidad de la política).

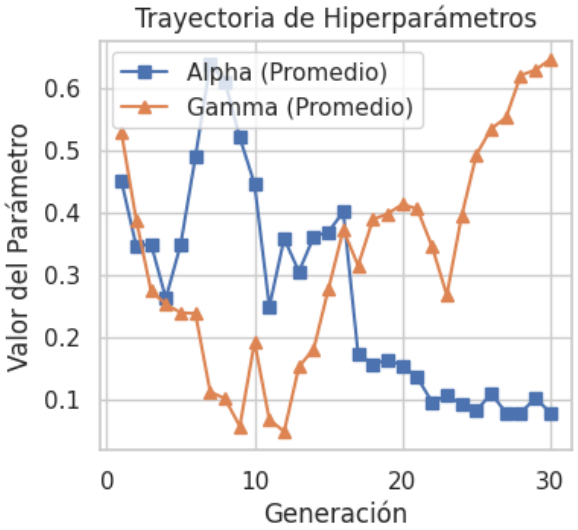
C. Resultados obtenidos

La Evolución del Fitness por Generación muestra que el Mejor Fitness (línea verde) oscila consistentemente entre un mínimo cercano a 1.22 y un máximo de 1.48, mientras que el Fitness Promedio de la población (línea azul discontinua) se mantiene estable en un rango de 0.98 y 1.21, con una media general alrededor de 1.04. La banda de variación (sombra azul) se mantiene ancha, fluctuando entre 0.90 y 1.15 en la mayoría de las generaciones.

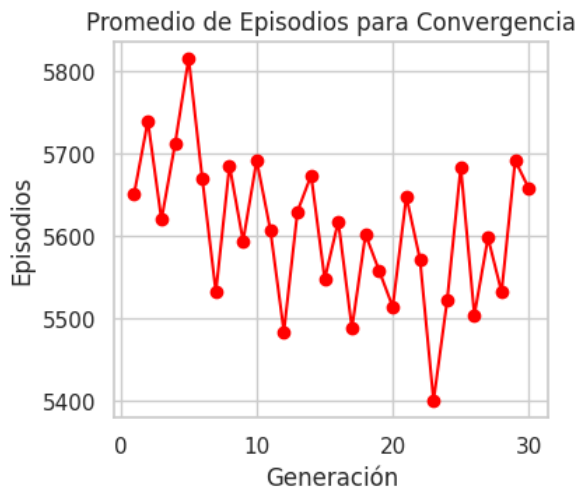


La Trayectoria de Hiperparámetros revela una clara tendencia evolutiva en los valores promedio de α y γ . Mientras que el valor promedio de Alpha comienza alrededor de 0.45 y disminuye progresivamente hasta estabilizarse cerca de 0.1 hacia la Generación 25. Por otro lado, el valor promedio de Gamma comienza alrededor de 0.50, experimenta caídas significativas, pero termina con un fuerte ascenso final, cerrando la simulación cerca de0.65 en la generación 30.

.



Finalmente, el Promedio de Episodios para Convergencia muestra una fluctuación general en la velocidad de aprendizaje, oscilando en un rango estrecho entre aproximadamente 5,400 y 5800 episodios. La media de convergencia de la población en todas las generaciones se sitúa alrededor de los 5,690 episodios, con el punto más rápido de convergencia registrado en aproximadamente 5,420 episodios y el más lento en cerca de 5,780 episodios.



También se puede apreciar la distribución de agentes con juego perfecto, en función de los Alpha y Gamma en la siguiente figura:

La densidad de estos agentes perfectos (áreas verdes) se concentra en al menos tres regiones principales del espacio de búsqueda:

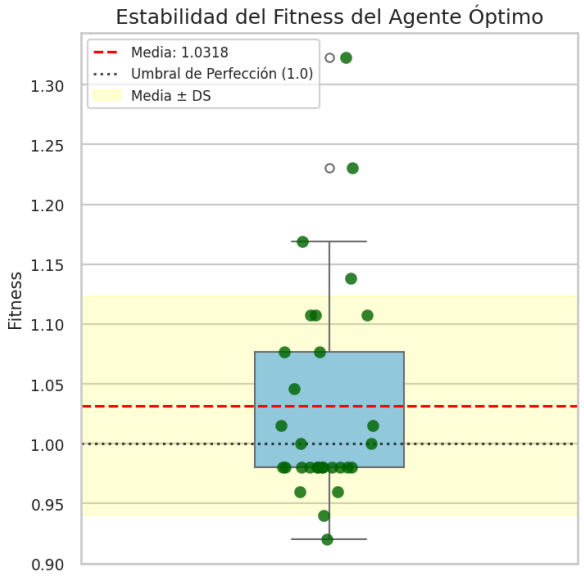
1. **Alpha bajo, Gamma medio:** Existe una alta concentración de agentes perfectos en el cuadrante inferior izquierdo, donde el valor promedio de Alpha se sitúa entre 0.1 y 0.3, y el valor promedio de Gamma se encuentra entre 0.4 y 0.7. Existe también un subgrupo denso en valores de Gamma bajos (cercanos a 0.1) en este rango de Alpha.
2. **Alpha alto, Gamma bajo:** Una segunda área densa de agentes perfectos se localiza en el cuadrante inferior derecho, con valores de Alpha entre 0.6 y 0.9, y Gamma muy bajos, alrededor de 0.0 a 0.2.
3. **Alpha medio, Gamma alto:** Se observa una tercera concentración menos densa en la parte superior izquierda, con valores de Alpha cerca de 0.1 a 0.3 y Gamma altos, entre 0.6 y 0.9.

Los puntos rojos dispersos indican que existen soluciones válidas en otras regiones, pero las áreas verdes muestran las combinacionee fueron más probables de generar un agente invencible durante la evolución.

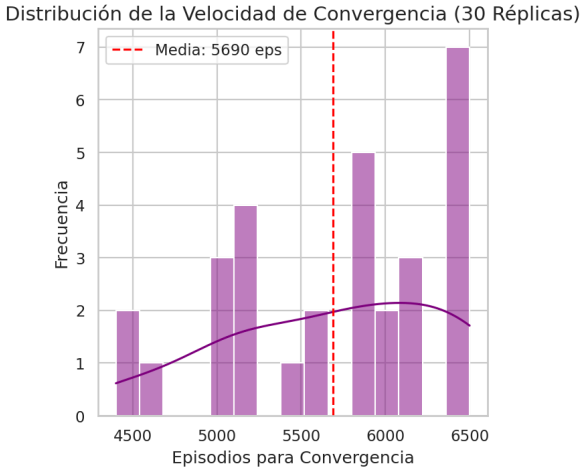
Luego de haber completado el torneo genético, el mejor agente global fue identificado en la Generación 14 del torneo, alcanzando un Fitness Máximo de 1.477 tras un total de 3400 episodios de entrenamiento. Los hiperparámetros óptimos asociados a este rendimiento son: Alpha igual a 0.6500, Gamma igual a 0.0300, y un Decay Rate de 0.006400. Este conjunto de valores fue sometido a una validación de robustez , obteniendo los siguientes resultados:

La Media del Fitness observada, indicada por la línea roja discontinua, es de 1.0318. El cincuenta por ciento central de los resultados, representado por la caja azul, se ubica predominantemente por encima del Umbral de Perfección de 1.0.

El pico máximo de Fitness alcanzado en alguna de las réplicas es de aproximadamente 1.32 (un punto atípico superior). El valor mínimo observado en la distribución se sitúa cerca de 0.92 (un punto atípico inferior). Se constata que múltiples réplicas lograron alcanzar el fitness de uno punto cero o superior en la siguiente figura:



Luego, se analizó la velocidad de convergencia de las réplicas, obteniendo los siguiente resultados:

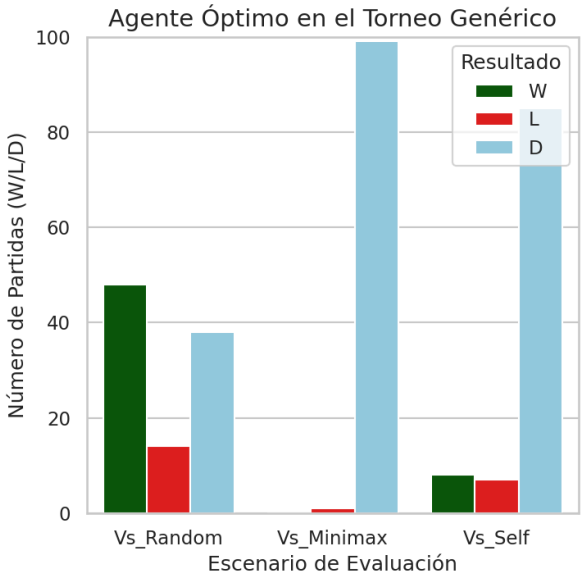


La distribución de las frecuencias muestra un patrón bimodal. Se identifica un Clúster Inferior (Rápido) donde un pico significativo de convergencia ocurre en el rango de 4500 a 5300 episodios. Existe un segundo y mayor pico, denominado Clúster Superior (Lento), registrado cerca del límite máximo del presupuesto, específicamente en el rango de 6000 a 6500 episodios. El rango total de observación para el proceso de convergencia abarca desde los 4000 a 6500 episodios.

Por último, una vez extraído los datos de consistencia del agente, se introduce en el torneo de desempeño, obteniendo los siguientes resultados:

Escenario	**Victorias (W)**	**Derrotas (L)**	**Empates (D)**	**Tasa de Éxito (W+D)**
Vs_Random	48	14	38	86.00%
Vs_Minimax	0	1	99	99.00%
Vs_Self	8	7	85	93.00%

Estos valores se pueden resumir en la siguiente figura:



D. Análisis

La metodología de optimización y validación se estructuró en tres fases secuenciales para garantizar que la solución encontrada fuera tanto óptima como robusta.

Para el torneo genético, los resultados de esta fase una fuerte tendencia del AG a favorecer un conjunto de hiper parámetros que lograba el mejor fitness (hasta 1.477 en algunos casos) en un tiempo de convergencia de tan solo 3400 episodios. La reducción progresiva de la tasa de mutación, de 0.30 a 0.01, fue clave para refinar estas soluciones.

Los hiper parámetros ganadores se probaron en 30 réplicas independientes para verificar la fiabilidad de los resultados. La media del Fitness Híbrido se mantuvo elevada en 1.0318, demostrando que, en promedio, el agente alcanzó la invencibilidad (Media $\geq$ 1.0). El Fitness Máximo observado fue de 1.32; sin embargo, la Desviación Estándar (DS) del fitness fue de 0.09140, un valor mayor a 0.05, lo que indica que el rendimiento de los agentes perfectos es sensible a las condiciones iniciales de cada entrenamiento. Respecto a la velocidad, el promedio de convergencia fue de 5,690 episodios, un ahorro significativo sobre el máximo de 6,500, lo que valida el objetivo de optimización de la velocidad.

El agente entrenado con los HPs validados fue probado en un torneo de tres escenarios con 100 partidas cada uno. Los resultados mostraron una alta especialización del agente: Mientras que contra Minimax mostró optimalidad cercana a la perfección (99% de éxito). Esto se debe a que el entrenamiento se centró intensamente en aprender a no perder contra el mejor oponente posible. La alta exposición al Minimax (o a jugadas que imitan al Minimax en el *self-play*) forzó al agente a converger hacia una política defensiva muy sólida, lo que resulta en casi todos los juegos siendo empates, tal como se espera de un agente bien entrenado contra un maestro perfecto.

Por otro lado, el rendimiento Vs_Random fue notablemente inferior, con un 14% de derrotas. Esto ocurre porque el agente nunca fue entrenado explícitamente para castigar errores aleatorios. Su política fue optimizada para ser "a prueba de Minimax" (es decir, evitar la pérdida a toda costa), no para ser "agresivo contra el error". Cuando el oponente comete un error que no es parte de la estrategia óptima del Minimax, el agente Q-Learning, prefiere asegurar el empate seguro antes que arriesgar una jugada que podría llevar a una derrota inesperada.

Finalmente, el escenario Vs_Self arrojó un 93% de éxito (o estabilidad), lo que indica que el agente es muy consistente cuando juega contra una política idéntica a la suya. Las derrotas en este escenario son mínimas y probablemente se deben a la inestabilidad inherente al auto-juego con parámetros no totalmente perfectos.

El rendimiento es coherente con el diseño del entrenamiento: el agente se especializó en la supervivencia contra la excelencia, y no en la explotación agresiva de la mediocridad.

V. LIMITACIONES DE LOS ALGORITMOS

El Q-learning enfrenta limitaciones notables, especialmente la escalabilidad en entornos con espacios de estado-acción grandes debido al tamaño de la Q-table que debe almacenar. Si el número de estados y acciones es muy elevado, la tabla se vuelve

imprácticamente grande, haciendo el aprendizaje ineficiente. Además, otro desafío inherente es encontrar el equilibrio adecuado entre exploración y explotación.

Por su parte, los Algoritmos Genéticos (AG), aunque son útiles para problemas de optimización no lineales en espacios de búsqueda grandes, su rendimiento depende significativamente de la correcta configuración de sus operadores (como cruce y mutación) y la función de aptitud (fitness). Una mala definición puede llevarlos a converger prematuramente a un óptimo local en lugar del óptimo global, o a tardar demasiado en encontrar una solución aceptable.

Cuando se comparan o combinan en un contexto como los torneos genéticos, las debilidades se hacen evidentes:

- **Q-learning en el Torneo:** Si el "individuo" en el torneo genético fuera un agente Q-learning (o sus parámetros), su aptitud (evaluada a través de su rendimiento en el entorno del torneo) estaría limitada por la carga computacional y la tasa de aprendizaje inherente al Q-learning. Un agente con un espacio de estado/acción muy grande podría no converger lo suficientemente rápido para obtener una aptitud alta antes de ser seleccionado o descartado.
- **Algoritmos Genéticos en el Torneo:** El torneo genético en sí mismo es un mecanismo de selección que busca fomentar la supervivencia de los mejores. La limitación aquí recae en el AG: si la función de aptitud que evalúa el rendimiento del agente es ruidosa o si los operadores genéticos no exploran el espacio de soluciones de manera efectiva, el torneo seleccionará individuos que parecen buenos pero no lo son, o descartará aquellos que podrían llevar al óptimo con más generaciones.

VI. REPOSITORIO

https://github.com/V-enekoder/tic-tac-AI-QLearning.git

VII. BIBLIOGRAFÍA

[1] – Russell, S. J., & Norvig, P. (2004). *Inteligencia Artificial. Un Enfoque Moderno* (2da ed.). Pearson Education. [Documento en línea]. Disponible en: https://luismejias21.wordpress.com/wp-content/uploads/2017/09/inteligencia-artificial-un-enfoque-moderno-stuart-j-russell.pdf

[2] Paniccia, M. (Prof.). (*s.f.*). *Q-Learning_De_Cero_a_Maestro .[Material de Curso]*. Universidad Nacional Experimental de Guayana (UNEG).

[3]Berger-Tal, Oded; Nathan, Jonathan; Meron, Ehud; Saltz, David (22 April 2014). "The Exploration-Exploitation Dilemma: A Multidisciplinary Framework". *PLOS ONE*. **9** (4) e95693. Bibcode:2014PLoSO...995693B. doi:10.1371/journal.pone.0095693.

[4] Pygame Community. (*s.f.*). *Welcome to Pygame*. [Documento en línea]. Disponible en: https://www.pygame.org/

[5]S. Dreyfus (2002), 'Richard Bellman on the birth of dynamic programming' *Operations Research* 50 (1), pp. 48–51.

[6]Bellman, R.E. 1957. *Dynamic Programming*. Princeton University Press, Princeton, NJ. Republished 2003: Dover, ISBN 0-486-42809-5.